



FACULTAD DE INGENIERÍA

Organización y Arquitectura de Computadores

MINI PROYECTO FASE 1

Diferencia de Temperaturas: Implementación en Lenguaje Ensamblador MIPS

INTEGRANTES: VALENTINA ANDRADE, SASKIA BRITO

PROFESOR: ALLAN ROBERTO AVENDAÑO SODARIO

PERIODO ORDINARIO II

Samboyondón, 22 de Septiembre del
2026

TABLA DE CONTENIDO

<i>1. Objetivo</i>	<i>3</i>
<i>2. Escenario asignado</i>	<i>3</i>
<i>a. Datos almacenados en memoria</i>	<i>3</i>
<i>3. Código fuente en MIPS</i>	<i>3</i>
<i>4. Explicación de solución</i>	<i>4</i>
<i>5. Resultados Esperados</i>	<i>5</i>
<i>6. Evidencia de Ejecución</i>	<i>5</i>
<i>7. Conclusión</i>	<i>6</i>
<i>Referencias</i>	<i>7</i>

1. Objetivo

Aplicar instrucciones de acceso a memoria, operaciones aritméticas y de comparación en lenguaje ensamblador MIPS mediante la resolución de un escenario práctico, comprendiendo el flujo de los datos desde la memoria hacia los registros, su procesamiento en la unidad central de procesamiento (CPU) y el almacenamiento de los resultados nuevamente en la memoria.

2. Escenario asignado

Se nos asignó el escenario “Diferencias de Temperatura”: se registran las temperaturas de dos momentos del día, mañana y tarde. El programa debe calcular la diferencia mañana-tarde como la diferencia entre tarde-mañana, almacenando ambos resultados en la memoria.

```
mañana = 22
tarde = 30
```

a. Datos almacenados en memoria

<i>Variable</i>	<i>Significado</i>	<i>Valor Inicial</i>	<i>Tipo</i>
dato1	Temperatura de la mañana	22	.word
dato2	Temperatura de la tarde	30	.word
resultado1	mañana - tarde	0 (inicial)	.word
resultado2	tarde - mañana	0 (inicial)	.word

3. Código fuente en MIPS

```
#Utilice las operaciones mul, sub ($t0 - $t1), sub ($t1 - $t0),
div, rem, seq y sne, según corresponda.
#Use las operaciones de carga (lw) y almacenamiento de datos
(sw).
#Utilice únicamente las variables en la columna datos
almacenados en memoria.

#Diferencia de temperaturas. Se registran las temperaturas de
dos momentos del día.
#Calcule tanto mañana - tarde como tarde - mañana y almacene
ambas diferencias.
#mañana = 22; tarde = 30
```

```

.data
dato1:      .word 22
dato2:      .word 30
resultado1: .word 0
resultado2: .word 0

.text
.globl main

main:
# PASO 1: Carga de datos desde la memoria RAM hacia los
registros de la CPU
    # Cargar datos desde memoria
    # lw ...
    lw $t0, dato1
    lw $t1, dato2
# PASO 2: Procesamiento aritmético en la Unidad
Aritmético-Lógica (ALU)
    # Realizar las operaciones correspondientes
    # ...
    sub $t2, $t0, $t1
    sub $t3, $t1, $t0
# PASO 3: Almacenamiento de los resultados procesados en la
memoria RAM
    # Guardar resultados en memoria
    # sw ...
    sw $t2, resultado1
    sw $t3, resultado2
# PASO 4: Finalización limpia de la ejecución
    li $v0, 0
    jr $ra

```

4. Explicación de solución

memoria → registros → CPU → memoria

- **Carga (lw):** las instrucciones `lw $t0, dato1` y `lw $t1, dato2` transfieren los valores almacenados en las direcciones de memoria `dato1` y `dato2` hacia los registros temporales `$t0` y `$t1`, dejando disponibles en la CPU los valores 22 (mañana) y 30 (tarde).
- **Procesamiento (sub):** la ALU ejecuta dos restas independientes. La instrucción `sub $t2, $t0, $t1` calcula mañana – tarde ($22 - 30 = -8$) y la instrucción `sub $t3, $t1, $t0`

calcula tarde – mañana ($30 - 22 = 8$). No se requieren las operaciones mul, div, rem, seq ni sne porque el escenario asignado solo exige comparar magnitudes mediante una resta en ambos sentidos, sin multiplicaciones, divisiones ni pruebas de igualdad.

- **Almacenamiento (sw):** las instrucciones sw \$t2, resultado1 y sw \$t3, resultado2 devuelven los resultados calculados en la CPU hacia las direcciones de memoria resultado1 y resultado2, dejando el resultado del programa persistido en el segmento .data.
- **Finalización:** li \$v0, 0 seguido de jr \$ra retorna el control desde main hacia la rutina de arranque del simulador, que finaliza la ejecución del programa.

5. Resultados Esperados

<i>Instrucción</i>	<i>Registro/Memoria afectada</i>	<i>Valor Resultante</i>
lw \$t0, dato1	\$t0	22
lw \$t1, dato2	\$t1	30
sub \$t2, \$t0, \$t1	\$t2	-8
sub \$t3, \$t1, \$t0	\$t3	8
sw \$t2, resultado1	resultado 1 (memoria)	-8
sw \$t3, resultado2	resultado2 (memoria)	8

6. Evidencia de Ejecución

mipsy web

Load

Save

Run

Reset

Kill

Step Back

Step Next

Download

MIPS Docs

About

source

decompiled

data

Untitled

```

1  Utilice las operaciones mul, sub ($0 - $t1), sub ($t1 - $t0), div, rem, seq y sne, según corresponda.
2  Use las operaciones de carga (lw) y almacenamiento de datos (sw).
3  Utilice únicamente las variables en la columna datos almacenados en memoria.
4
5  #Diferencia de temperaturas. Se registran las temperaturas de dos momentos del día.
6  Calcule tanto mañana - tarde como tarde - mañana y almacene ambas diferencias.
7  $mañana = $t1; tarde = $0
8
9  .data
10 dato1: .word 12
11 dato2: .word 30
12 resultado1: .word 0
13 resultado2: .word 0
14
15 .text
16 .globl main
17
18 main:
19 # Cargar datos desde memoria
20 # lw ...
21 lw $t0, dato1
22 lw $t1, dato2
23
24 # PASO 2: Procesamiento aritmético en la Unidad Aritmético-Lógica (ALU)
25
26 # Realizar las operaciones correspondientes
27 # ...
28 sub $t2, $t0, $t1
29 sub $t3, $t1, $t0
30
31 # PASO 3: Almacenamiento de los resultados procesados en la memoria RAM
32 # Guardar resultados en memoria
33 # sw ...
34 sw $t2, resultado1
35 sw $t3, resultado2
36
37 # PASO 4: Finalización limpia de la ejecución
38 li $v0, 0
39 jr $ra

```

used registers

all registers

Read Write

Register

Value

⊖ ⊖	\$zero	0
⊖ ⊖	\$at	0x1001000c
⊖ ⊖	\$v0	17
⊖ ⊖	\$a0	0
⊖ ⊖	\$a1	0
⊖ ⊖	\$t0	22
⊖ ⊖	\$t1	30
⊖ ⊖	\$t2	-8
⊖ ⊖	\$t3	8
⊖ ⊖	\$t0p	0x7fffffff
⊖ ⊖	\$t0p	0x7fffffff
⊖ ⊖	\$ra	0x0000000c

I/O

Mipsy Output - (0)

I/O

mipsy web

Load

Save

Run

Reset

Kill

< Step Back

> Step Next

Download

MIPS Docs

About

source

decompiled

data

Untitled

```

main:
0x00400000 [0x3c011001] lui    $at, 4097          ; [20] lw $t0, dat01
0x00400004 [0x34210000] ori    $at, $at, 0
0x00400008 [0x8c200000] lw     $t0, ($at)
0x0040000c [0x3c011001] lui    $at, 4097          ; [21] lw $t1, dato2
0x00400010 [0x34210004] ori    $at, $at, 4
0x00400014 [0x8c290000] lw     $t1, ($at)
0x00400018 [0x01095022] sub     $t2, $t0, $t1          ; [25] sub $t2, $t0, $t1
0x0040001c [0x01285822] sub     $t3, $t1, $t0          ; [26] sub $t3, $t1, $t0
0x00400020 [0x3c011001] lui    $at, 4097          ; [30] sw $t2, resultado1
0x00400024 [0x34210000] ori    $at, $at, 8
0x00400028 [0xac2a0000] sw      $t2, ($at)
0x0040002c [0x3c011001] lui    $at, 4097          ; [31] sw $t3, resultado2
0x00400030 [0x3421000c] ori    $at, $at, 12
0x00400034 [0xac2b0000] sw      $t3, ($at)
0x00400038 [0x20020000] addi   $v0, $zero, 0          ; [33] li $v0, 0
0x0040003c [0x03e00000] jr      $ra              ; [34] jr  $ra

```

used registers

all registers

Read	Write	Register	Value
		\$zero	0
		\$at	0x1001000c
		\$v0	17
		\$a0	0
		\$a1	0
		\$t0	22
		\$t1	30
		\$t2	-8
		\$t3	8
		\$sp	0x7fffffff
		\$fp	0x7fffffff
		\$ra	0x8000000c

I/O

7. Conclusión

El escenario de diferencia de temperaturas se resolvió aplicando el ciclo memoria-registro-CPU-memoria propia de la arquitectura MIPS. Con solo dos instrucciones lw, dos sub y dos sw fue posible calcular ambas diferencias solicitadas y devolver los resultados al segmento de datos.

Referencias

mipsy. (s/f). Edu.au. Recuperado el 22 de septiembre de 2026, de <https://cgi.cse.unsw.edu.au/~cs1521/mipsy/>

West, Z. (2020, noviembre 27). MIPS Store Word (sw) vs. Load Word (lw). alparithms. Alpharithms. <https://www.alparithms.com/mips-store-word-sw-vs-load-word-lw-475521/>